DESIGN ON PRIORITY AMBULANCE DISPATCH SYSTEM

1. The Problem:

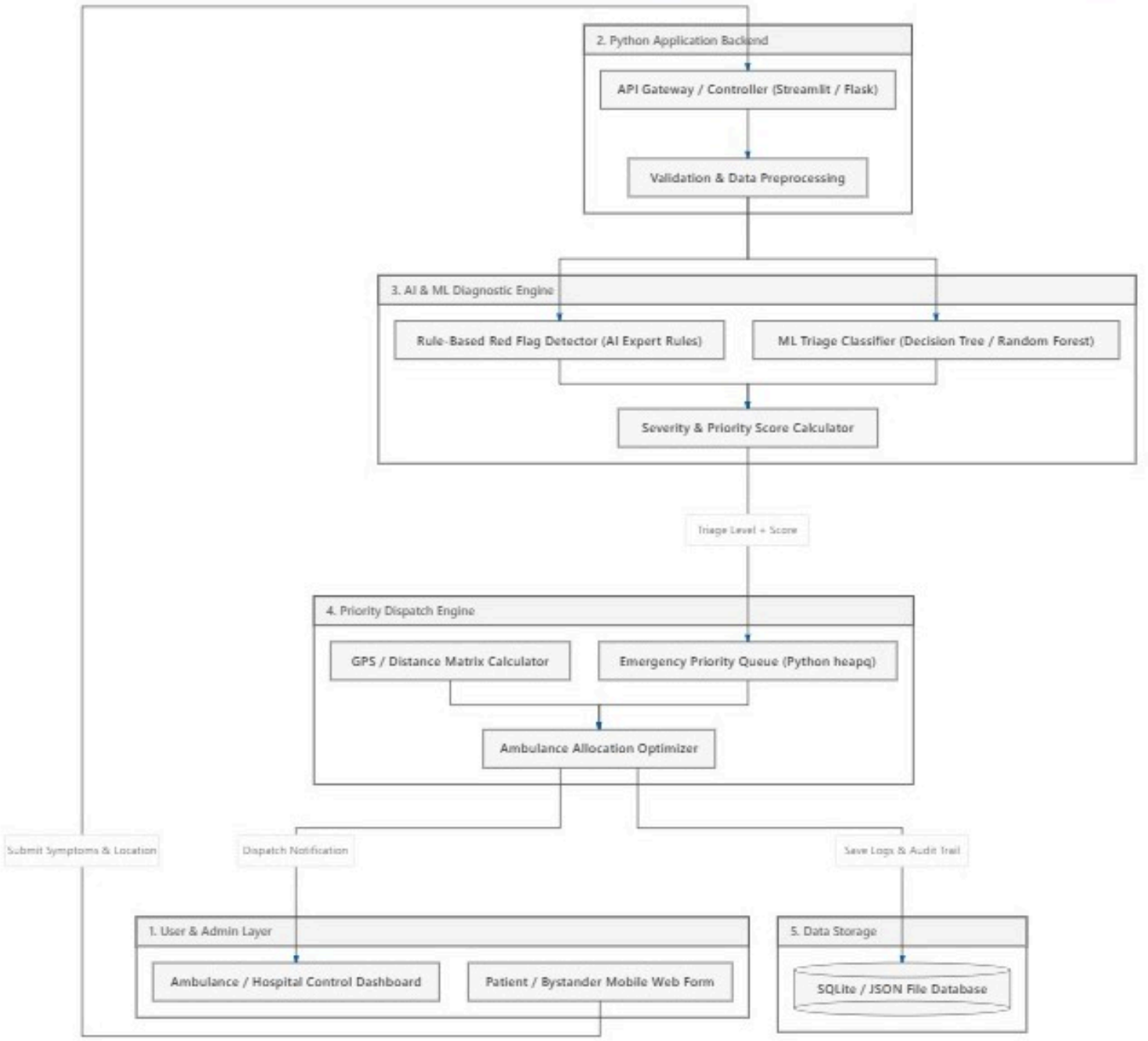
In medical emergencies, response time is critical. Traditional emergency dispatch systems often operate on a "first-come, first-served" basis or rely on human operators who may struggle to quickly assess multiple simultaneous emergencies. A patient suffering a minor cut might call five minutes before someone experiencing cardiac arrest, leading to suboptimal ambulance allocation.

2.The Solution:

An automated, intelligent emergency tracker where:

- Patients or their family members report symptoms, vital signs, and location.
- An **AI/ML Engine** evaluates the severity score and tags the emergency with a triage colour code:
  - 🔴 **Red (Priority 1 - Critical / Life-Threatening)**: Heart attack, severe trauma, stroke, unconsciousness.
  - 🟡 **Yellow (Priority 2 - Urgent)**: Fractures, moderate asthma attacks, high fever with complications.
  - 🟢 **Green (Priority 3 - Non-Urgent)**: Minor sprains, cuts, mild stomach ache.
- A **Priority Dispatch Algorithm** in Python routes available ambulances to **Red Flag** cases first, factoring in both medical urgency and geographic proximity.

3. High-Level System Architecture



4. Understanding the Roles: Python, AI, and ML

Technology Component	Role in this Project	Beginner Implementation Details
Python (Core Backbone)	Handles the application logic, data structures, math calculations, and UI framework.	• Manages data intake using dictionaries/pandas. • Implements a `PriorityQueue` using Python's built-in `heapq` module. • Calculates distance using the Haversine formula.
AI (Expert Rules / Reasoning)	Instant, deterministic red-flag overrides that can never fail or wait for statistical probabilities.	• Rule-based medical safety net: *If patient is unconscious OR has chest pain with breathing difficulty → Immediate 🔴 Red Flag regardless of other metrics.*
ML (Statistical Pattern Learning)	Predicts severity and triage level for complex, ambiguous cases where multiple vitals interact.	• A **Decision Tree** or **Logistic Regression** trained on patient vital parameters (Heart Rate, Systolic BP, Oxygen Saturation, Age, Pain Scale).

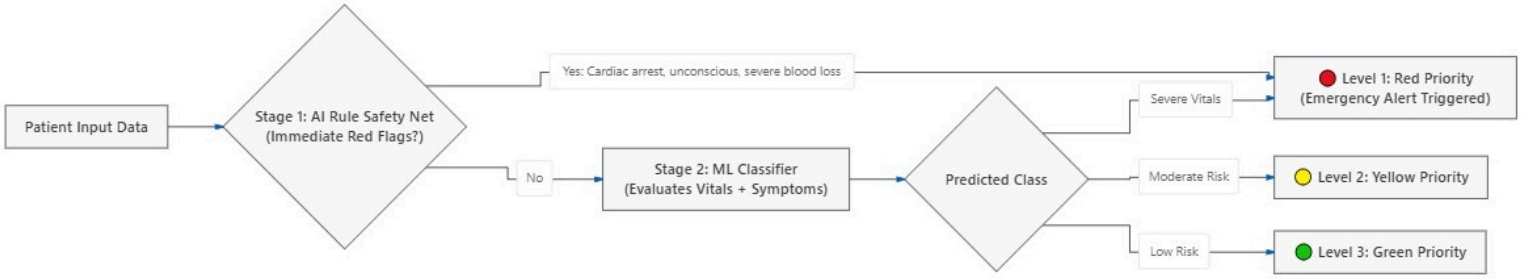
5. Detailed Module Breakdown

Module 1: Patient / Bystander Intake Portal

- **Input Parameters**:
  - **Patient Identity**: Name, Age, Gender, Contact number.
  - **Location**: GPS coordinates or manually selected sector/address.
  - **Vital Signs**(if known):
    - Heart rate (BPM)
    - Blood Pressure (Systolic / Diastolic)
    - Oxygen Saturation (SpO2 %)
    - Body Temperature (∘F)
  - **Symptoms Checklist**: Chest pain, shortness of breath, loss of consciousness, uncontrolled bleeding, severe burn, nausea, etc.
  - **Pain Scale**: Slider from 1 (mild) to 10 (unbearable).

Module 2: The AI & ML Diagnostic Triage Engine

The triage logic operates in a 2-stage pipeline:



1. **Stage 1 (AI Hard-Rules)**: Immediate checks based on clinical guidelines (e.g., Glasgow Coma Scale indicators or Manchester Triage System).
2. **Stage 2 (ML Severity Scoring)**:
   - A trained Scikit-Learn model outputs a probability vector: $P(\text{Red}), P(\text{Yellow}), P(\text{Green})$.
   - Final Priority Score formula: $\text{Priority Score} = w1 \cdot \text{Triage Level} + w2 \cdot \text{Pain Scale} + w3 \cdot \text{Waiting Time}$

Module 3: Priority Dispatch & Routing Engine

Ambulances should not just be assigned to who is closest; they must be assigned to the most critical patient first:

- The system maintains an active **Priority Queue** sorted by:
  1. Priority Category (Red > Yellow > Green).
  2. Patient Severity Score (Higher score = higher priority).
  3. Waiting time (preventing starvation of Yellow patients).
- **Proximity Calculation**: Calculates the Haversine distance between each available ambulance and the high-priority incident.
- **Dispatch Action**: Picks the nearest available vehicle and transitions its status to DISPATCHED.

Module 4: Hospital / Dispatch Control Dashboard

- Live map or grid view showing:
  - Active incoming emergency requests coloured by urgency (🔴, 🟡, 🟢).
  - Locations and availability status of all fleet ambulances (Available, En Route, At Hospital).
  - Audio/visual alert whenever a new 🔴 **Red Flag** case enters the queue.

6. Data Design & Schema

For a beginner-friendly Python implementation, a relational SQLite database or structured JSON files can be used:

Table 1: `emergencies`

Field Name	Data Type	Description
`incident_id`	Integer (Primary Key)	Unique emergency identifier
`patient_name`	Text	Name of patient
`contact`	Text	Emergency contact number
`latitude`	Real / Float	Location latitude
`longitude`	Real / Float	Location longitude
`heart_rate`	Integer	Heart beats per minute
`systolic_bp`	Integer	Systolic blood pressure
`spo2`	Real / Float	Blood oxygen percentage
`pain_level`	Integer	Pain score (1 to 10)
`primary_symptom`	Text	Major reported symptom
`triage_level`	Text	'RED', 'YELLOW', or 'GREEN'
`priority_score`	Real / Float	Calculated urgency score
`status`	Text	'WAITING', 'DISPATCHED', 'RESOLVED'
`created_at`	Timestamp	Time when emergency was logged

Table 2: `ambulances`

Field Name	Data Type	Description
`ambulance_id`	Text (Primary Key)	E.g., 'AMB-101'
`driver_name`	Text	Driver / Paramedic contact
`current_lat`	Real / Float	Current GPS Latitude
`current_lng`	Real / Float	Current GPS Longitude
`status`	Text	'AVAILABLE', 'ON_MISSION', 'MAINTENANCE'
`assigned_incident_id`	Integer (Foreign Key)	Current active task

7. Recommended Technology Stack for a First-Year Student

Layer	Recommended Tool	Why It Fits a Beginner
Language	**Python 3.10+**	Readable, clean syntax, massive support for AI and data science.
Frontend / Web UI	**Streamlit**	Build interactive web dashboards and forms purely in Python in under 100 lines of code without needing HTML/CSS/JavaScript.
Machine Learning	**Scikit-Learn**	Clean API for training simple classifiers (`DecisionTreeClassifier` or `RandomForestClassifier`).
Data Processing	**Pandas & NumPy**	Fast data filtering, manipulation, and array mathematics.
Priority Queue	**Python** `heapq`	Standard library priority heap for queue management.
Database	**SQLite3**	Zero-setup file-based SQL database built directly into Python.
Geospatial / Maps	**Folium / Geopy**	Easy creation of interactive maps directly from Python.

8. Sample Algorithmic Workflow

